

06/10 ISW

📅 Date	06/10/2026
☰ Materia	ISW

Aseguramiento $\neq$ calidad del testing

QA $\neq$ Testing

QA $\rightarrow$ Conjunto de cosas que hay que hacer durante el proceso de construcción del producto $\rightarrow$ Que se tenga calidad involucra un conjunto de acciones que apuntan a dicho objetivo

Calidad $\rightarrow$ Subjetivo (percepción personal), se valida contra los requerimientos, adecuación de un determinado producto o servicio en base a las necesidades del cliente o usuario que solicitó (nivel que cumple dichas expectativas).

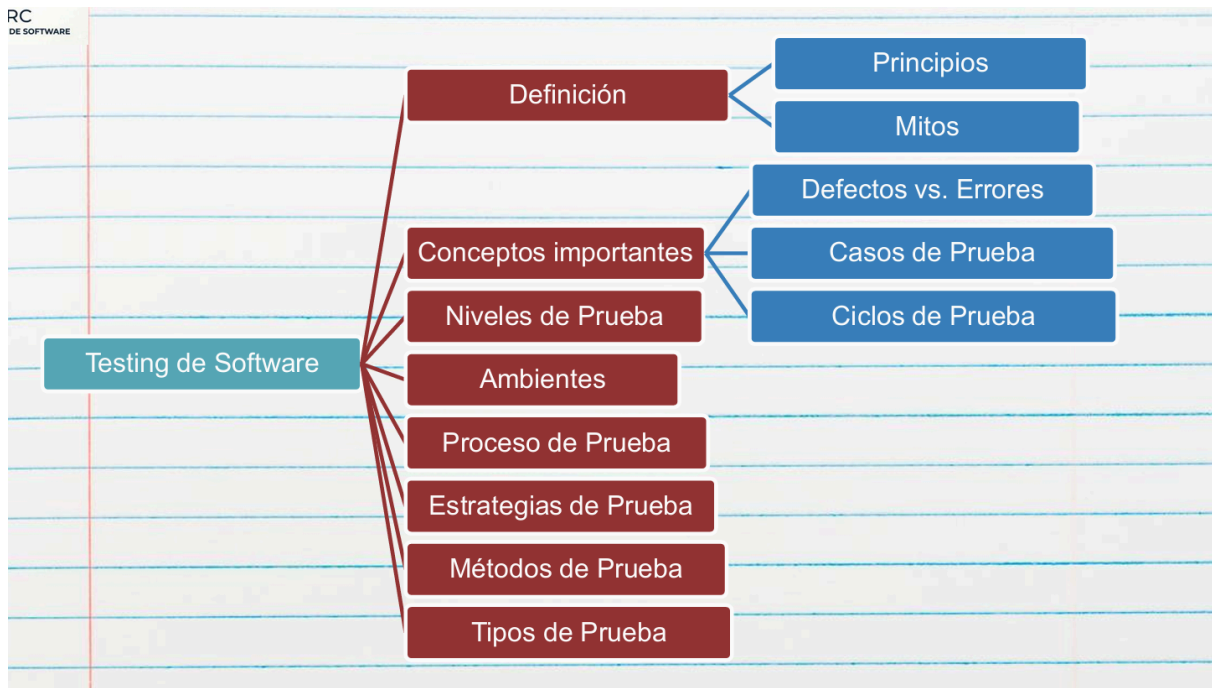
Expectativa $\neq$ Necesidad

Expectativas son cosas que el cliente QUIERE pero no manifiesta $\rightarrow$ Req. no funcionales (generalmente)

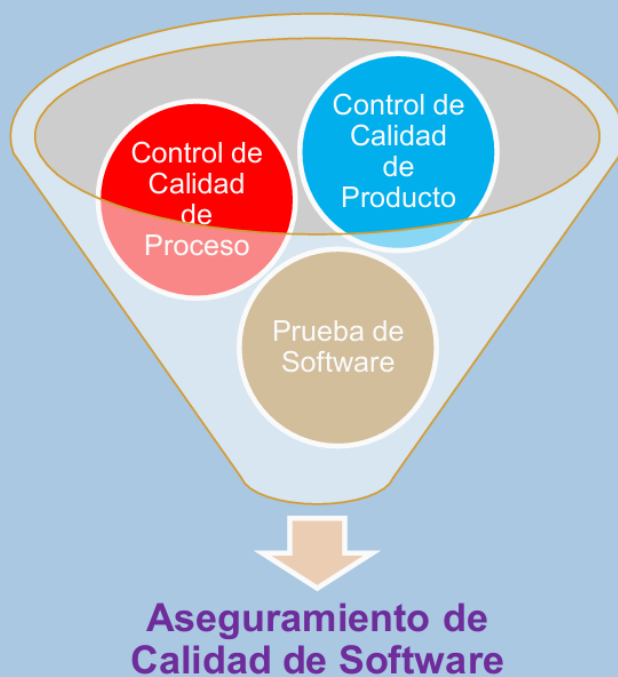
Lo más difícil es definir el producto a crear $\rightarrow$

Propósito del testing $\rightarrow$ Encontrar defectos cuya presencia se asume en el SW

- El desafío es encontrar los errores
- NO se puede garantizar ni asegurar la calidad, ni que no existan defectos.
- El proceso de testing hace evidentes los defectos ENCONTRADOS, no todos
- Proceso de control contra ALGO $\rightarrow$ ERS, requerimientos
- MITO $\rightarrow$ TESTING NO ASEGURA NI CERTIFICA CALIDAD NI EVITA DEFECTOS
- Realidad $\rightarrow$ Hace visible defectos que se asumen
-



Administración de Configuración



Aseguramiento de calidad → Acompaña al desarrollo de SW, puede generar un testing que FRACASE (lo cual no es necesariamente malo)

Testing exitoso → Encuentra errores

Workflow de prueba (PUD)

Requerimientos → Analisis → Diseño → Implementacion → PRUEBA → Despliegue

Validacion → El cliente lo lleva a cabo, probar que el producto sea correcto

Verificacion → Que funcione correctamente

Error ≠ Defecto

Error → Queda en el contexto de la misma fase donde aparecio (por ejemplo, si aparece en la etapa de requerimiento, no pasa de fase, es lo deseable y mas barato)

Defecto → Pasa de etapa → Testing encuentra DEFECTOS en el codigo, porque son introducidos en las etapas anteriores

Testing es PARTE de lo que se hace para asegurar calidad

Se debe llevar un buena administracion de configuracion para llevar control sobre los ICS

El 30% a 50% del costo del SW

1º nivel de testing → Unitario, quien construyo, usar TDD (automatizar pruebas)

Pruebas estaticas y dinamicas

Caja negra → Dinamicas, se ejecuta el SW

Caja blanca → Miran el codigo, es estatico

El testing no asegura calidad ni esta se puede inyectar al final

- Una vez definidos los requerimientos de calidad, tengo que tener en cuenta que:
 - La calidad no puede "inyectarse" al final
 - La calidad del producto depende de tareas realizadas durante todo el proceso
 - Detectar errores en forma temprana ahorra esfuerzos, tiempo, recursos
 - La calidad no solamente abarca aspectos del producto sino también del proceso y como éstos se pueden mejorar, que a su vez evita defectos recurrentes.
 - El testing NO puede asegurar ni calidad en el software ni software de calidad

Siempre es mas barato prevenir que corregir luego

Falla ≠ Error

Falla → Mal funcionamiento del sistema, consecuencia de una introducción de un defecto.

NO todos los defectos provocan FALLAS, el defecto es HUMANO, que provocan en el sistema FALLAS. HAY defectos que NO causan FALLAS DE MAL FUNCIONAMIENTO

Ej. Defectos de diseño

CONCEPTOS: DEFECTOS, SEVERIDAD Y PRIORIDAD	
Severidad	Prioridad
1 – Bloqueante	1 – Urgencia
2 – Crítico	2 – Alta
3 – Mayor	3 – Media
4 – Menor	4 – Baja
5 – Cosmético	

Escalas impares → Humano tiende a tirar al medio

Severidad → Cuan grave es el defecto, cuanto afecta al producto, el cliente NO puede marcar o determinar la severidad, determinada por el equipo de testing
→ Debe haber una DEFINICION ESCRITA para evitar conflictos entre desarrolladores y testing → El daño que le causa el defecto al SW

Bloqueante → No se puede utilizar el sistema

Crítico → No puede cumplir el objetivo con una funcionalidad determinada, no hay alternativa de conseguir el objetivo

Mayor → La funcionalidad no anda pero hay distintas formas de llevar a cabo el objetivo

Menor → Afecta a la funcionalidad pero no impide su cumplimiento, pueden ser advertencias

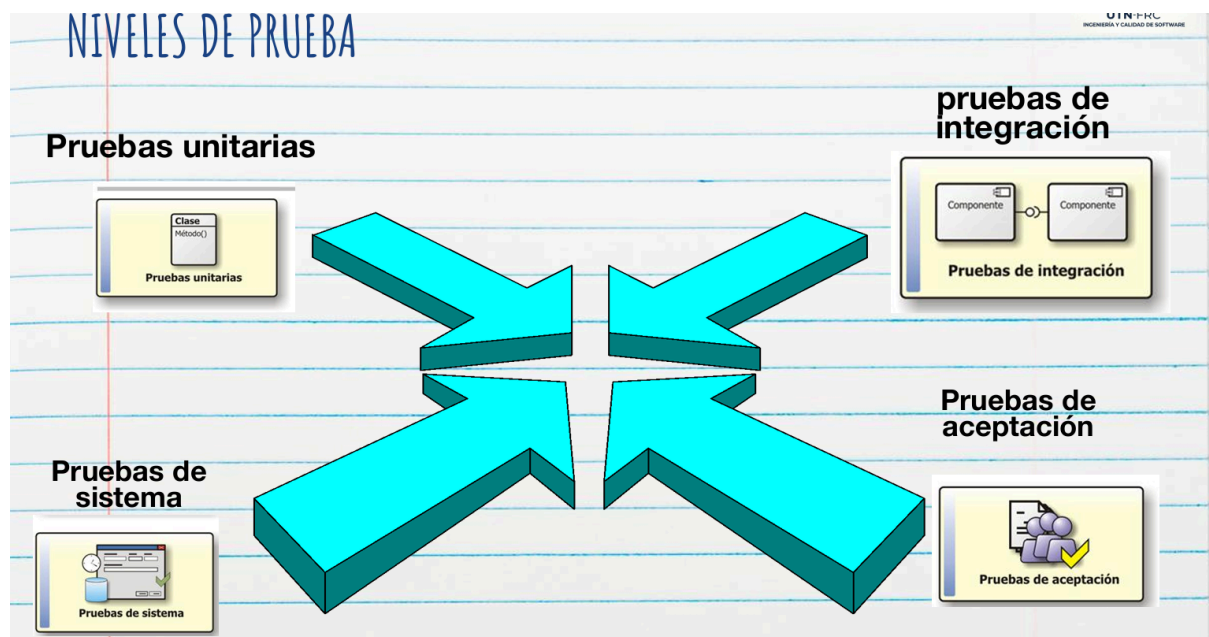
Cosmético → Ortografía, usabilidad, disposición de componentes

Prioridad DEFINIDA por el PO, el cliente (peor caso) → Vinculados a SLAs, el enfoque AGIL busca evitar estas cuestiones de tener problemas con el cliente

Urgente → NO SE PUEDE ESPERAR! Manifestacion de deseo

La prioridad indica hasta cuanto se puede esperar para la correccion del defecto

Pueden haber errores graves con prioridad media o baja



Niveles de prueba

Estos niveles poseen un orden de ejecucion, divididos como:

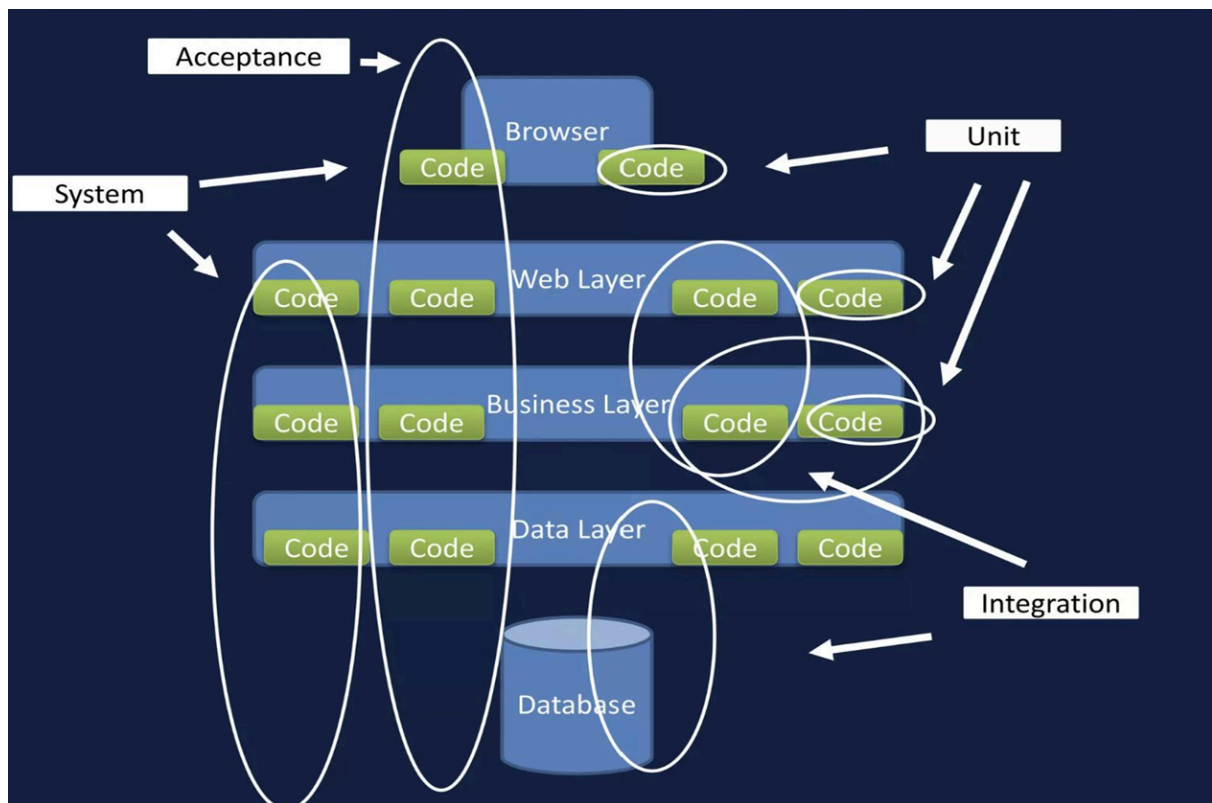
Primer nivel: Pruebas unitarias, integradas en la tecnica en TDD y llevadas a cabo por los desarrolladores; excepcion de la regla, quien lo crea lo prueba, en el resto debe ser alguien aparte. → Se llevan a cabo en el WF de implementacion

Segundo nivel: Pruebas de integracion o de interfaces, el foco es probar si el funcionamiento entre dos componentes es correcto, el resultado es un build (una version del producto, empaquetado, que posee varios componentes). Teoricamente, se llevan a cabo en el workflow de prueba, pero en la realidad es dificil que esto sucede, por ello se lleva a cabo implementacion; si se trabaja con la integracion continua, se automatizan las pruebas de integracion

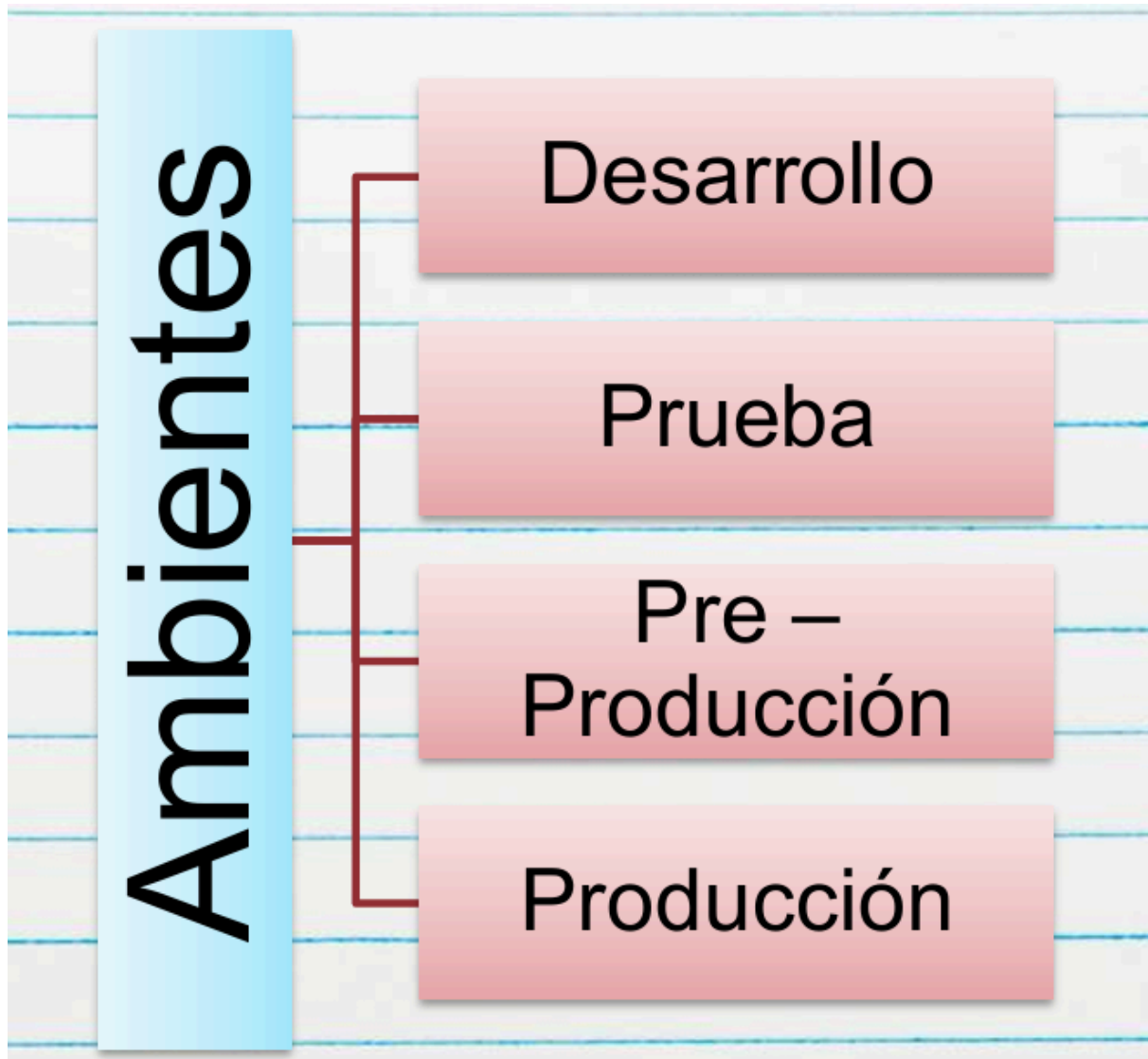
Tercer nivel: Pruebas de sistema, propiedad del WF de pruebas, conocidas tambien como pruebas de version. Debe ser llevada por alguien INDEPENDIENTE DE QUIEN LO CONSTRUYO. Pueden ser proveedores externos, empresas distintas, es deseable que no compartan jefe entre el

grupo de desarrollo y del testing para evitar aprietes (diferencia con testing agil)

Cuarto nivel: Pruebas de aceptacion de usuario (UAT, User Acceptance Test). Pruebas llevadas a cabo el usuario, dependen del nivel de aceptacion del producto. Se llevan a cabo en el despliegue



Pruebas de sistema → Gente de testing para evitar que el usuario encuentre defectos



Configuración del espacio de trabajo → Desarrollo y testing deben estar en ENTORNOS DISTINTOS

Entorno: Configuración del espacio de trabajo con el repositorio y todas las herramientas necesarias para el desarrollo o pruebas (SO, firewall, programas, etc etc) → Debe estar planificado en el plan de gestión de configuración

Importante porque

- Tema manejo de datos
- Problema de que en una máquina ande y en otra no
- Objetividad a la hora de hacer testing, espacio donde los desarrolladores no tengan injerencia; evitar que se hagan pruebas y correcciones al mismo tiempo sobre la misma versión

Desarrollo → Implementación

Prueba → Pruebas (duh)

Produccion → NO deben llevarse a cabo pruebas de aceptacion porque ES MUY PELIGROSO, puede afectar al funcionamiento del sistema

Pre-produccion → Ambiente que comparte las mismas características que el ambiente de produccion para llevar a cabo las pruebas de aceptacion.

No funcionales muy dependientes del HW, por ello ambos ambientes deben ser iguales, el mismo entorno

Se pueden resolver con la tecnica de Continuos Deployment

Casos de prueba → Set de instrucciones, paso a paso, que permiten llegar a un objetivo. Artefacto entregable del testing

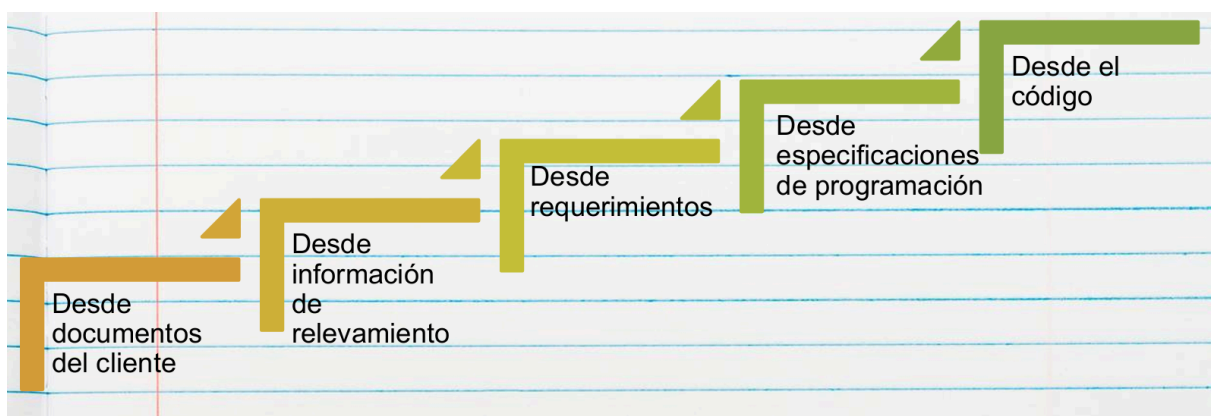
Para que algo sea un defecto, debe ser REPRODUCIBLE, a traves del set de instrucciones puede volverse a generar el mismo defecto

No se pueden crear pruebas exhaustivas, probarlo TODO → Reducir la cantidad de pruebas a diseñar, SIN reducir la calidad de las pruebas → Ejecutar una cantidad de pruebas que cierre en terminos de costos y beneficios y que lleve a un testing exitoso

Caja negra

- Valores limite
- Partición de equivalencia

Las pruebas las sacamos de las US



La validacion se hace contra lo documentado; Para la verificacion solamente se puede probar el codigo

La informacion para hacer los casos de prueba se deben sacar de los requerimientos

Proceso estandar para testing (Etapas)

Planificacion de pruebas → Diseño de Casos de prueba → Ejecucion →
Reporte de defectos (si no salen bien las pruebas, se repite la ejecucion)

El artefacto de la planificacion que sale es el plan de prueba (Estrategias, tipos de testing, ambientes y su configuracion, metodos)

En el diseño salen los casos de prueba y los datos de prueba

En la ejecucion entra la version del SW + los datos de prueba, cuyo resultado es el reporte y metricas.

La planificaicon y diseño, no es buena idea esperar al SW para crear las pruebas

La ejecucion y reportes tienen un periodo de tiempo, llamado ciclo de prueba. Puede terminar forzosamente si ocurre un defecto bloqueante. El flujo normal termina con el reporte de prueba hecho, el cual vuelve a desarrollo para llevar a cabo las correcciones, dando otra version. Cuando esta version llega, se incrementa otro ciclo (parte del 0 en adelante) → Lo mejor que puede ocurrir es que sean dos ciclos (el cero para detectar los defectos y el uno para correcciones validas) → Escenario ideal

Regresion → Pruebas de regresion, realizando la mismas pruebas que se usaron para el ciclo 0 para verificar la nueva version.

Pruebas sin regresion → Solo se verifican los defecto, estadistica dice que cada defecto que se corrige aparecen 3; no es bueno, no los detecta porque NO corre todos los casos de prueba → La automatizacion busca corregir esto

En agil el proceso cambia, porque hablamos de incrementos o funcionalidades; mientras que en el proceso tradicional son versiones del SW